Critical Thinking Questions

#1 |

Can data in memory be called a file? Explain.

No, data in memory cannot be called a file. Data in memory exists only while the program is running and it is represented as variables, objects, arrays, etc. The file is stored on permanent storage (like a hard disk or CD).

#2 |

Write the import statement required to access the File Class in an application.

Import java.io.*;

#3 |

Identify the error in the following statement:

File textFile = new File("c:\inventory.txt");

The path to the file is wrong because a single dash (\ ) in Java represents an escape character. Instead the double dash (\\ ) or reversed dash (/) should be used for the path, and it should look like this:

File textFile = new File("c:\\inventory.txt"); **OR** File textFile = new File("c:/inventory.txt");

#4 |

A) Which statement is used to write an exception handler?

catch

B) Write an exception handler to handle an IOException if a specified file name cannot be used to create a file. The exception handler should display appropriate messages to the user.

```
catch(IOException e)
    {
       System.out.println("File name cannot be used to create a file.");
       System.err.println("IOException: " + e.getMessage());
    }
```

#5 |

A) What is the name of the stream for displaying error messages

```
System.err.println("IOException: " + e.getMessage());
```

B) Where are these messages displayed?

Messages sent to System.err are typically displayed in the console.

#6 |

A) What does the file stream keep track of?

It keeps track of file position, which is the point where reading or writing last occurred.

B) What characters together make up a line terminator?

\n

#7 |

What two classes are used together to write data to a file?

```
Filewriter;
BufferedWriter;
```

#8 |

Write a statement to convert account balances that have been read from a text file to a double value and add them to totalBalance.

```
double totalBalance = 0;
        try
        {
          in = new FileReader(textFile);
          readFile = new BufferedReader(in);
          while ((lineOfText = readFile.readLine()) != null)
           {
                    String bal = lineOfText;
                    double balance = Double.parseDouble(bal);
                    totalBalance += balance;
            }
        }
```

#9 |

Explain the difference between object serialization and object deserialization.

Object serialization and object deserialization are opposite processes used in programming to store or transfer objects. Serialization is the process of converting an object into a format that can be stored or transmitted. Deserialization is the reverse process: converting stored/transmitted data back into an object.

#10 |

What interface must be implemented if objects of a class are to be written to a file.

If you want objects of a class to be written to a file (i.e., serialized), the class must implement the Serializable interface.